**Федеральное государственное бюджетное образовательное учреждение высшего образования**
**«Херсонский государственный педагогический университет»**

Институт информационных технологий

Кафедра программной инженерии и информационных технологий

# ПРАКТИЧЕСКАЯ РАБОТА
# по дисциплине:
# «Программирование графических процессоров»

## на тему:
## «Практическая работа 5. Работа с матрицами»

Выполнил:
Студент гр.: 12-25РПм
Трифонов Д.С.

Проверил
Преподаватель:
Алешов Е.В.

Оценка:_____
Дата: 26.10.2026

**Херсон, 2026**

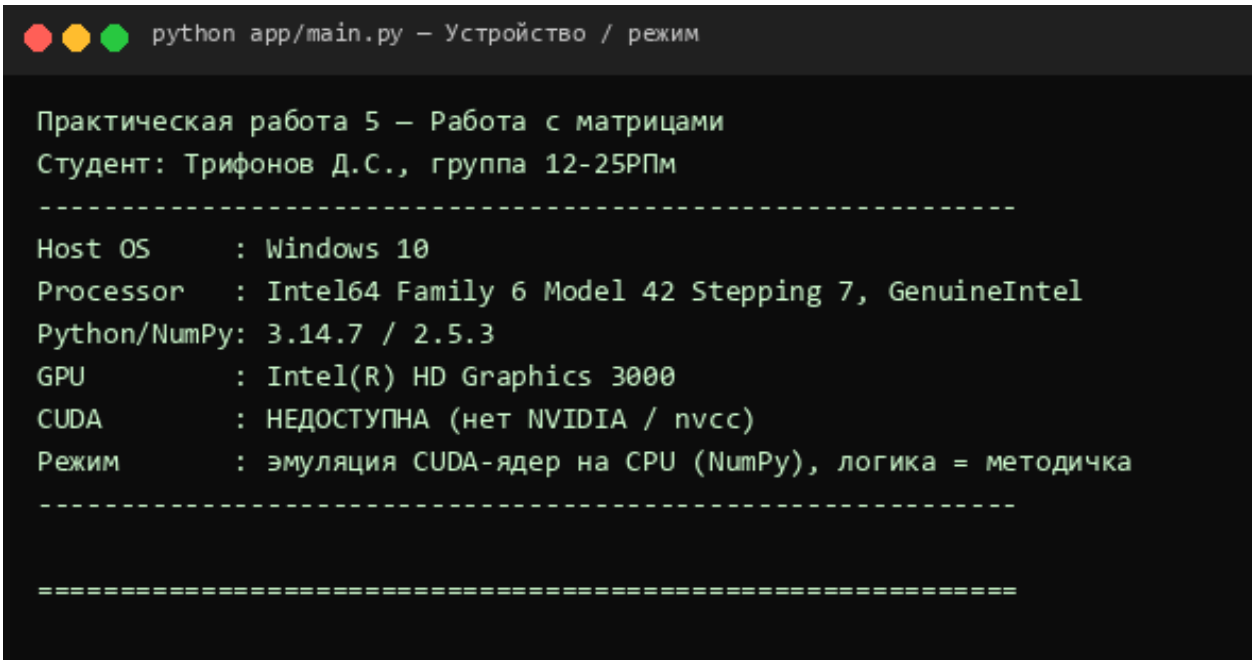
# ПРАКТИЧЕСКАЯ РАБОТА
## Практическая работа 5. Работа с матрицами

Цель работы: освоить операции над матрицами в модели CUDA (создание на устройстве, транспонирование, сложение, умножение) и выполнить контрольные задания методички.

### 1. Аппаратная платформа и выбор реализации

На компьютере установлен интегрированный GPU Intel HD Graphics 3000 (Sandy Bridge). Платформа CUDA требует видеокарту NVIDIA и toolkit (nvcc), которые отсутствуют. Драйвер Intel OpenCL на данной системе при сборке ядер зависает и экспонирует только CPU i5-2520M. Поэтому программа реализована на Python/NumPy с эмуляцией модели запуска CUDA: типы Dim3, буферы DeviceArray (аналог cudaMalloc), функция launch() повторяет схему kernel<<<grid, block>>>, а ядра сохраняют индексацию threadIdx/blockIdx из листингов методички.

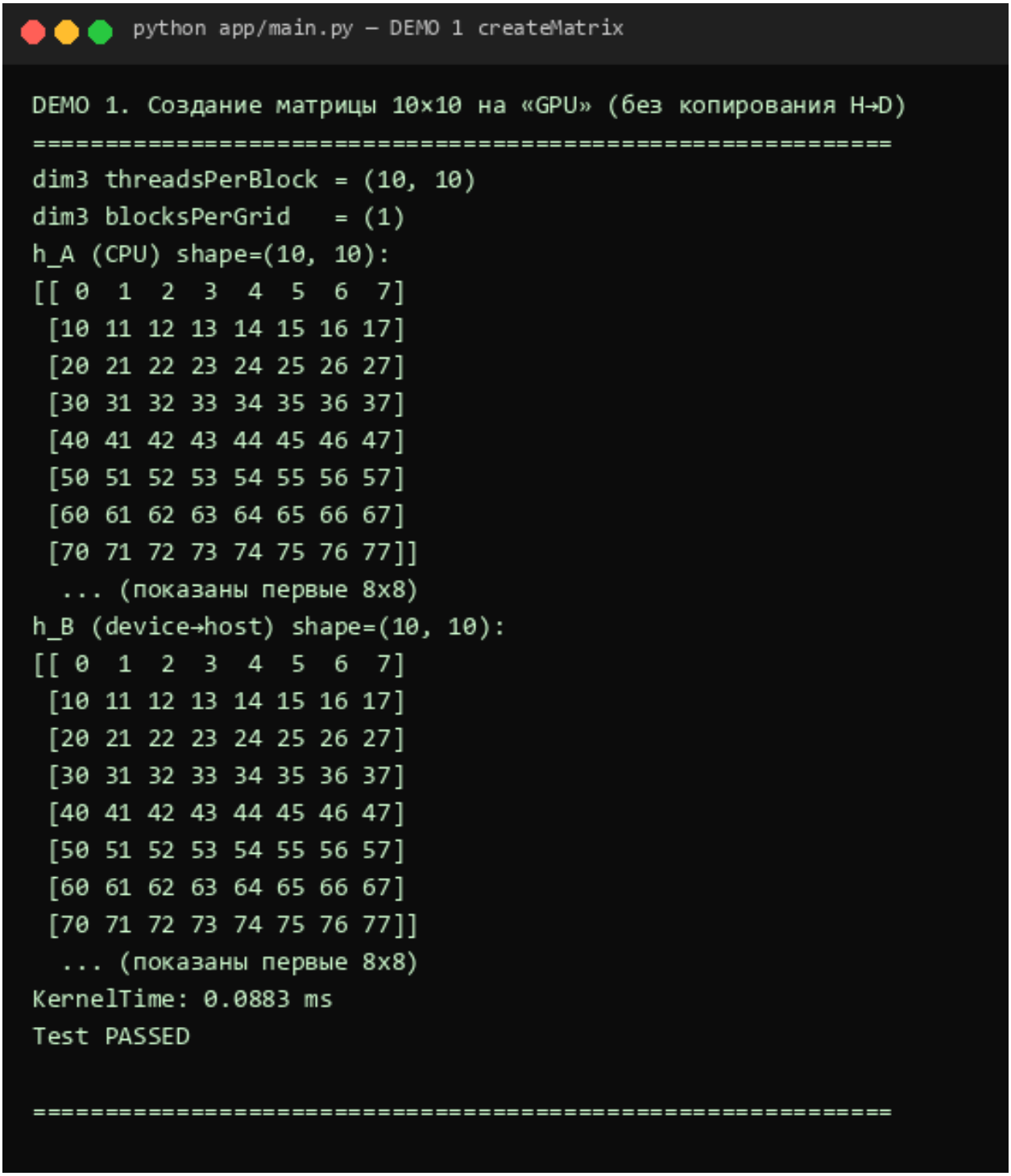
Рисунок 1 — Информация об устройстве и режиме выполнения




### 2. Демонстрация операций из методички

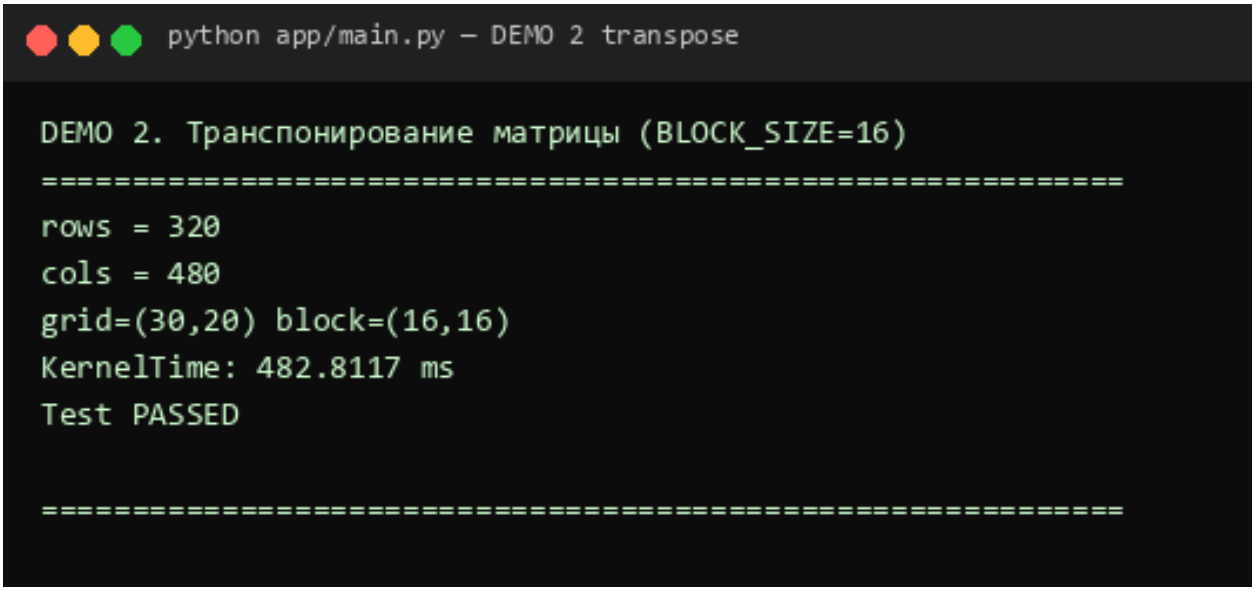
2.1. Создание матрицы 10×10 на устройстве без копирования H→D. Ядро createMatrix заполняет A[ty*n+tx] = 10*ty+tx; на CPU матрица строится двойным циклом. После копирования D→H матрицы совпадают.

Рисунок 2 — Создание и сравнение матриц A и B



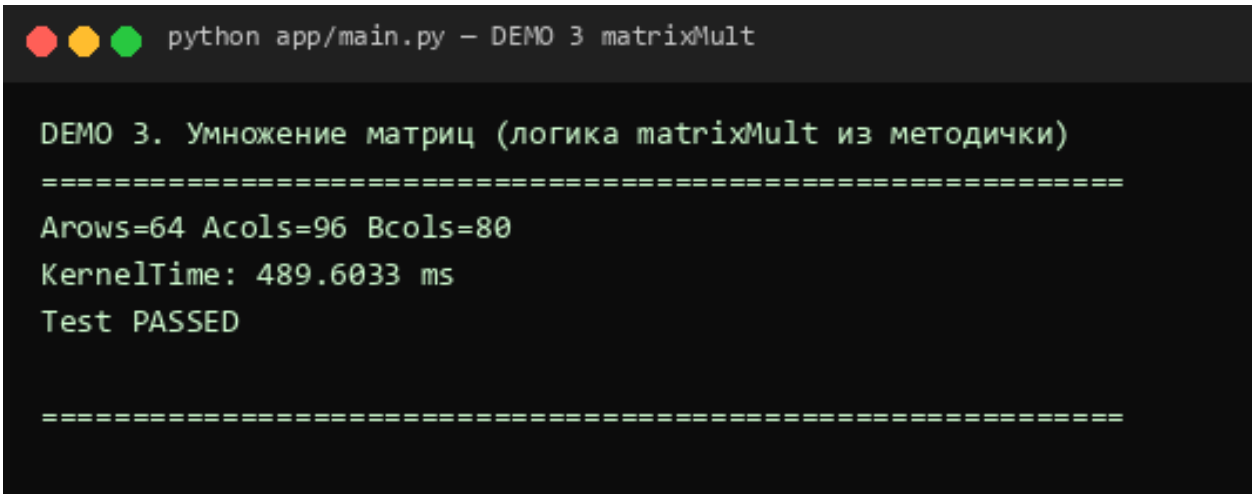

2.2. Транспонирование. Блоки 16×16, размеры матрицы приведены к кратным BLOCK_SIZE функцией toMultiple. Проверка: AT[j,i] == A[i,j].

Рисунок 3 — Транспонирование матрицы




2.3. Умножение матриц. Ядро matrixMult считает сумму по k для элемента C[i,j]. Результат сверяется с эталоном NumPy (допуск 1e-3).

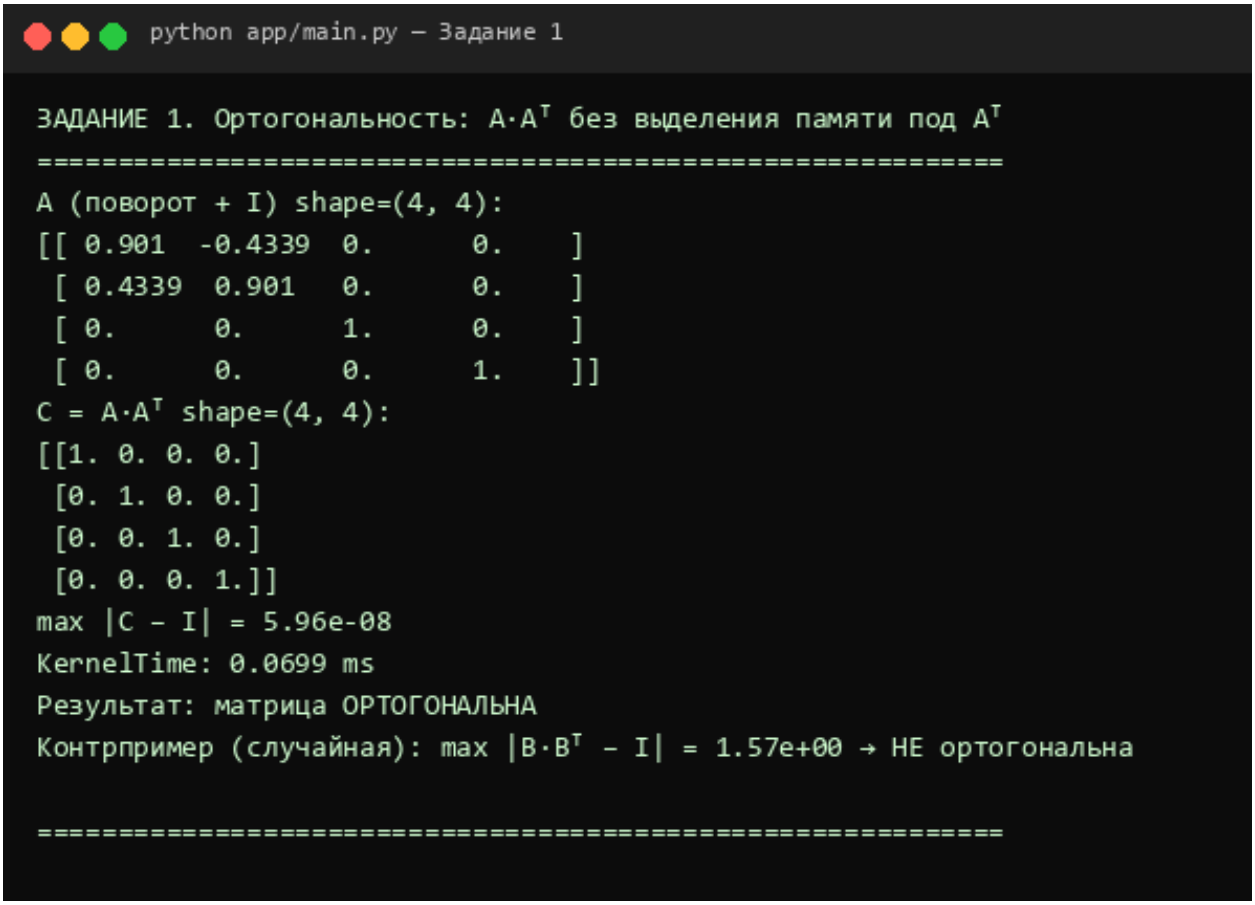
Рисунок 4 — Умножение матриц




## 3. Выполнение заданий практической работы

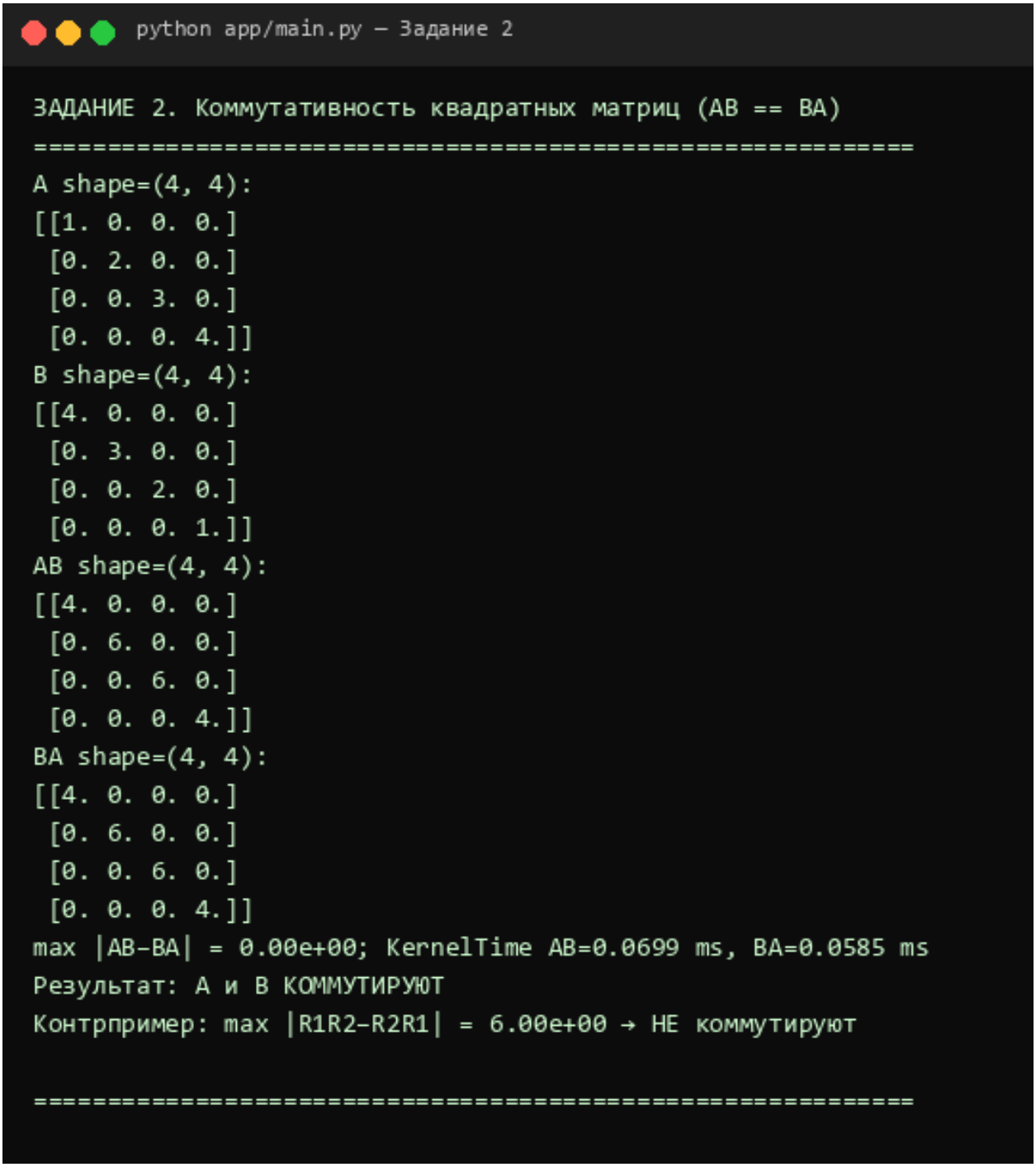
Задание 1. Проверка ортогональности квадратной матрицы умножением на транспонированную без выделения дополнительной памяти под $A^T$. В ядре mult_by_transpose элемент $(A^T)[k,j]$ читается как A[j,k]. Для ортогональной матрицы (блок поворота 2D внутри $I_4$) получено $C \approx I$ (ошибка ~1e-7). Случайная матрица даёт большую ошибку.

Рисунок 5 — Проверка ортогональности




Задание 2. Проверка коммутативности: AB и BA считаются двумя запусками ядра умножения. Диагональные A и B коммутируют (max|AB−BA|=0). Контрпример из верхне/нижнетреугольных матриц показывает, что произведение зависит от порядка.

Рисунок 6 — Проверка коммутативности




Задание 3. Сложение матриц M×N, заданных на хосте как вектор векторов (list[list] / float a[M][N]). Массивы копируются в DeviceArray, на «устройстве» выполняется ядро matrixAdd, результат возвращается на хост и снова представляется как list[list].

Рисунок 7 — Сложение матриц как вектор векторов

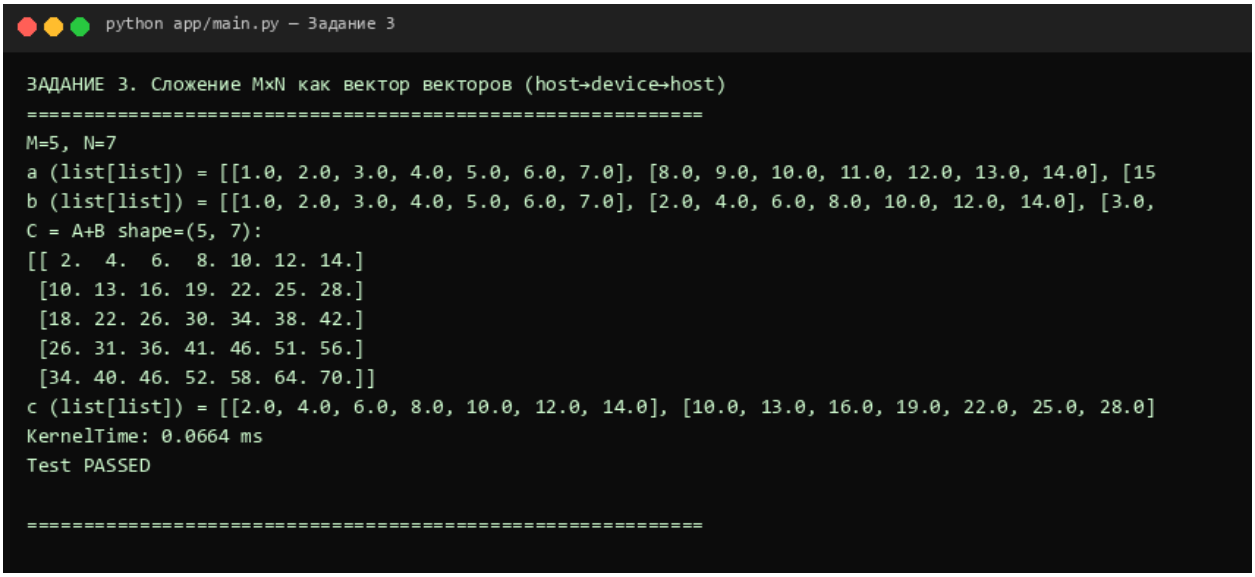
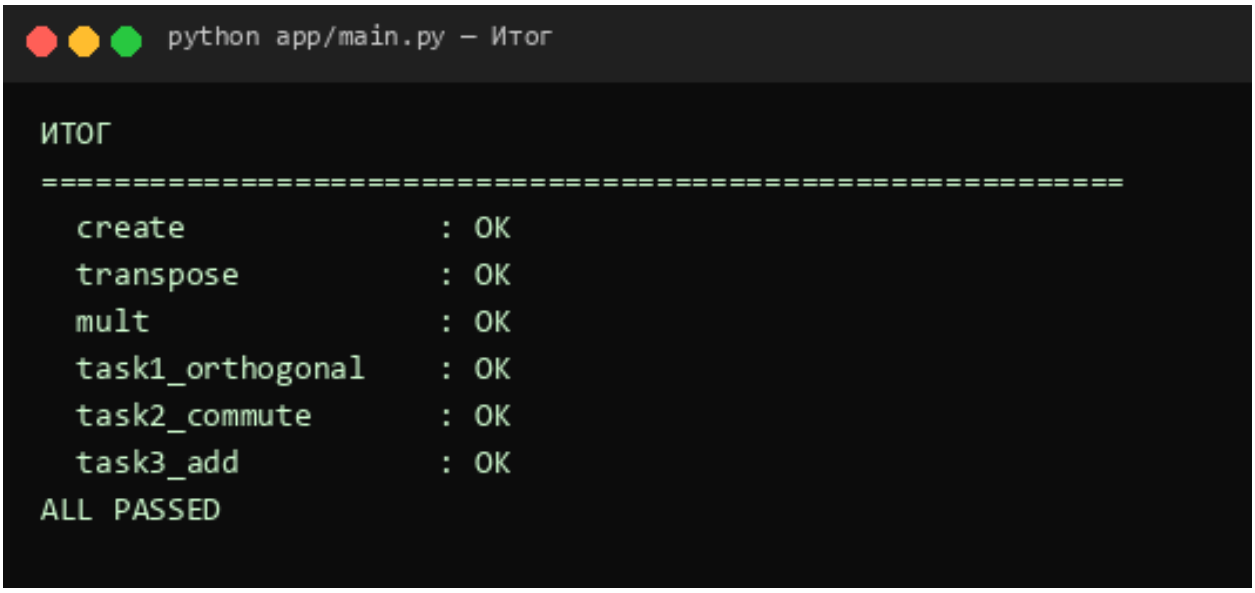



Рисунок 8 — Итог выполнения всех тестов




## 4. Фрагменты программного кода (app/main.py)

Ниже приведены ключевые части: Dim3/запуск, ядра и задания.

```
class Dim3:
    x: int = 1
    y: int = 1
    z: int = 1

    def __post_init__(self):
        self.x = int(self.x) if self.x else 1
        self.y = int(self.y) if self.y else 1
        self.z = int(self.z) if self.z else 1


class DeviceArray:
    """Буфер «на устройстве» (память хоста, семантика cudaMalloc)."""

    def __init__(self, data: np.ndarray | None = None, shape=None, dtype=np.float32):
        if data is not None:
            self.data = np.array(data, copy=True)
        else:
```

```
            self.data = np.zeros(shape, dtype=dtype)

    @property
    def nbytes(self):
        return self.data.nbytes

    @property
    def shape(self):
        return self.data.shape

# ...

def kernel_create_matrix(bx, by, tx, ty, bdx, bdy, A: DeviceArray, n: int):
    # A[threadIdx.y * n + threadIdx.x] = 10 * threadIdx.y + threadIdx.x
    x = bx * bdx + tx
    y = by * bdy + ty
    if x < n and y < n:
        A.data[y, x] = 10 * y + x

# ...

def kernel_mult_by_transpose(bx, by, tx, ty, bdx, bdy, A, C, n):
    """C = A * A^T без буфера A^T: элемент (A^T)[k,j] = A[j,k]."""
    j = bx * bdx + tx
    i = by * bdy + ty
    if i < n and j < n:
        s = 0.0
        for k in range(n):
            s += float(A.data[i, k]) * float(A.data[j, k])
        C.data[i, j] = s

# ...

def task1_orthogonal():
    print("\n" + "=" * 60)
    print("ЗАДАНИЕ 1. Ортогональность: A·Aᵀ без выделения памяти под Aᵀ")
    print("=" * 60)
    theta = np.pi / 7
    c, s = float(np.cos(theta)), float(np.sin(theta))
    A = np.eye(4, dtype=np.float32)
    A[0, 0], A[0, 1] = c, -s
    A[1, 0], A[1, 1] = s, c
    print_matrix("A (поворот + I)", A)

    n = 4
    d_A = DeviceArray(A)
    d_C = cuda_malloc((n, n))
    ms = launch(kernel_mult_by_transpose, Dim3(1), Dim3(n, n), d_A, d_C, n)
    C = cuda_memcpy_dtoh(d_C)
    err = float(np.max(np.abs(C - np.eye(n, dtype=np.float32))))
    print_matrix("C = A·Aᵀ", C)
    print(f"max |C − I| = {err:.2e}")
    print(f"KernelTime: {ms:.4f} ms")
    orth = err < 1e-5
    print("Результат: матрица ОРТОГОНАЛЬНА" if orth else "Результат: НЕ ортогональна")

    B = np.random.RandomState(0).rand(4, 4).astype(np.float32)
    d_B, d_Cb = DeviceArray(B), cuda_malloc((n, n))
    launch(kernel_mult_by_transpose, Dim3(1), Dim3(n, n), d_B, d_Cb, n)
    err2 = float(np.max(np.abs(cuda_memcpy_dtoh(d_Cb) - np.eye(n))))
    print(f"Контрпример (случайная): max |B·Bᵀ − I| = {err2:.2e} → НЕ ортогональна")
```

```
    return orth and err2 > 1e-3

# ...

def task3_add_vv():
    print("\n" + "=" * 60)
    print("ЗАДАНИЕ 3. Сложение M×N как вектор векторов (host→device→host)")
    print("=" * 60)
    M, N = 5, 7
    a = [[float(i * N + j + 1) for j in range(N)] for i in range(M)]
    b = [[float((i + 1) * (j + 1)) for j in range(N)] for i in range(M)]
    print(f"M={M}, N={N}")
    print("a (list[list]) =", a)
    print("b (list[list]) =", b)

    h_A = np.array(a, dtype=np.float32)
    h_B = np.array(b, dtype=np.float32)
    d_A, d_B = DeviceArray(h_A), DeviceArray(h_B)
    d_C = cuda_malloc((M, N))
    threads = Dim3(N, M)
    blocks = Dim3(1, 1)
    ms = launch(kernel_add, blocks, threads, d_A, d_B, d_C, N, M)
    h_C = cuda_memcpy_dtoh(d_C)
    ok = np.allclose(h_C, h_A + h_B)
    print_matrix("C = A+B", h_C)
    print("c (list[list]) =", h_C.tolist())
    print(f"KernelTime: {ms:.4f} ms")
    print("Test PASSED" if ok else "Test FAILED")
    return ok
```

## 5. Ответы на контрольные вопросы

1. Чем отличается создание матрицы на GPU от создания на CPU с последующим копированием? При создании на GPU (ядро createMatrix) данные сразу формируются в памяти устройства параллельно нитями, без cudaMemcpy Host→Device. Вариант «создать на CPU и скопировать» требует шинного трансфера и последовательной инициализации на хосте; на больших матрицах копирование становится узким местом.

2. Где возникают потери производительности при операциях над матрицами? Основные потери: (а) копирование Host↔Device; (б) невыровненный доступ к глобальной памяти (особенно при транспонировании без shared memory); (в) низкая арифметическая интенсивность у сложения (memory-bound); (г) неоптимальный размер grid/block и границы, не кратные warp/блоку; (д) в 2D-массивах указателей лишние разыменования по сравнению с плоским 1D-массивом a[i*N+j].

## 6. Вывод

Выполнены демонстрации create/transpose/mult и три задания методички. Из-за отсутствия NVIDIA CUDA на Intel HD Graphics 3000 использована CPU-эмуляция с идентичной индексацией нитей. Все проверки завершились успешно (ALL PASSED). Проект: app/main.py.